Plateforme GRC de gestion des vulnérabilités

enrichie par la détection des menaces
et l'automatisation de la remédiation

Guide de projet détaillé & vérifié — étape par étape

Stage · ~6 semaines · à distance · 100% open source

Inventaire → Scan OpenVAS → Scoring → Détection → Remédiation auto → Dashboard → Rapport

Faisabilité matérielle et licences vérifiées (voir §2)

Sommaire

1. Vue d'ensemble & périmètre validé

2. Faisabilité & licences — VÉRIFIÉES (à lire avant de commencer)

3. Ce que tu construis vraiment (la place d'OpenVAS)

4. Architecture générale

5. Stack technique (versions & licences)

6. Modèle de données

7. Le moteur de scoring de risque

8. Le pipeline OpenVAS (install → scan → import)

9. Détection & remédiation automatique (la brique de la prof)

10. Planning détaillé semaine par semaine

11. Dashboard, KPI & reporting PDF

12. Pièges à éviter & bonnes pratiques

13. Extensions possibles

14. Checklist de démarrage

1. Vue d'ensemble & périmètre validé

Le projet est une **application web de gestion des risques cyber (GRC)** dont la mission tient en une phrase : aider une entreprise à savoir si elle est en sécurité, et sinon, quoi corriger en priorité. Le périmètre retenu résulte d'un compromis validé : on part d'une base GRC solide (gestion des vulnérabilités) que l'on **enrichit d'une couche de détection des menaces et d'automatisation de la remédiation**.

> **Titre du projet (validé)**
> « Conception et développement d'une plateforme GRC de gestion des vulnérabilités, enrichie par la détection des menaces et l'automatisation de la remédiation. »

Le pipeline complet

> Inventaire des actifs → Scan OpenVAS → Vulnérabilités (CVE + CVSS) → Moteur de scoring contextualisé → **Détection** (vuln critique corrélée à un actif majeur) → **Remédiation automatique** (alerte + action prioritaire créée) → Dashboards KPI → Rapport PDF

La brique ajoutée par l'encadrante

L'encadrante souhaite renforcer la partie « détection dynamique des menaces + automatisation de la remédiation » — par exemple : dès qu'une vulnérabilité critique est corrélée à un actif majeur, le système déclenche automatiquement une alerte ou une action. **Bonne nouvelle : cette brique s'implémente au niveau applicatif** (corrélation + règles + déclenchement), sans avoir à monter l'infrastructure réseau Zero Trust du sujet initial. On reste donc dans un périmètre réaliste pour la durée du stage. Voir §9 pour l'implémentation honnête et démontrable.

> **Honnêteté technique — important**
> La « détection » ici = corrélation applicative (un scan révèle une vuln critique sur un actif majeur). La « remédiation automatique » = automatisation par règles (alerte + création d'action). C'est réel et utile, mais il ne faut PAS le survendre comme un moteur d'IA de détection réseau temps réel — un technicien le verrait immédiatement. Décrit tel qu'il est, c'est solide.

2. Faisabilité & licences — VÉRIFIÉES

Cette section répond à la vraie question : « est-ce que ça tourne sur ma machine, et est-ce que tout est gratuit/légal ? ». Les chiffres ci-dessous proviennent de la **documentation officielle Greenbone** et des dépôts officiels des outils — pas d'estimations au doigt mouillé.

2.1 — Ton matériel vs les besoins réels

Ressource	OpenVAS min.	OpenVAS recommandé	Ton setup (×2 PC)
CPU	2 cœurs	4 cœurs	Core Ultra 5 (largement OK)
RAM	4 GB	8 GB	32 GB / machine — confortable
Disque	20 GB	60 GB	prévoir ~60–80 GB libres

Verdict matériel : tu es **très au-dessus** des besoins. OpenVAS/Greenbone recommande 8 GB de RAM ; tu en as 32 par machine. Tu peux même dédier une machine à Greenbone + la cible de scan, et l'autre au développement de l'application. Aucun souci de puissance.

> **Point clé souvent oublié — OpenVAS tourne sous Linux**
>
> Greenbone Community Edition s'installe sous Linux (Debian, Ubuntu 24.04, Fedora, CentOS). Si tes PC sont sous Windows, tu fais tourner Greenbone dans une **VM Linux (VirtualBox)** ou via **WSL2** — avec 32 GB de RAM tu alloues 8–12 GB à la VM sans problème. Ton application web, elle, se développe sur n'importe quel OS.

2.2 — Licences : tout est open source et gratuit

Composant	Licence	Coût	Vérifié
Greenbone / OpenVAS Community Ed.	GPL	Gratuit	doc officielle
python-gvm (API GMP)	GPL v3	Gratuit	PyPI officiel
gvm-tools	GPL v3	Gratuit	doc officielle
Python / FastAPI / Flask	PSF / MIT / BSD	Gratuit	—
PostgreSQL	PostgreSQL Lic.	Gratuit	—
Streamlit / Chart.js	Apache 2.0 / MIT	Gratuit	—
ReportLab (PDF)	BSD	Gratuit	—
lxml (parsing XML)	BSD	Gratuit	—
Docker / docker-compose	Apache 2.0	Gratuit	—
Metasploitable 2 / DVWA (cibles)	Libres (labo)	Gratuit	—

> **Verdict licences**
>
> **Zéro coût de licence, zéro dépendance propriétaire.** Toute la stack est publiable sur GitHub. Le seul piège classique : si tu utilises **VMware** comme hyperviseur, ce n'est pas open source → prends **VirtualBox** (GPL) à la place. Tout le reste est clean.

2.3 — Les vrais points d'attention (à savoir avant, pas pendant)

- **Le premier chargement des « feeds » prend du temps.** Au premier démarrage, Greenbone télécharge sa base de tests de vulnérabilités (VT), les données SCAP/CVE, CERT... Ça peut prendre de 30 min à plusieurs heures selon la connexion, et occupe l'essentiel des 20–60 GB de disque. C'est normal — lance-le tôt et laisse tourner, ne panique pas si le premier scan n'est pas dispo immédiatement.
- **Greenbone CE est un stack de ~20 conteneurs Docker.** C'est lourd mais orchestré automatiquement par le docker-compose officiel. Sur 32 GB ça passe sans souci.
- **La doc officielle précise que ce n'est pas destiné à la production.** Pour un projet de stage / démo, c'est parfait et parfaitement légitime — mentionne juste que c'est un environnement de labo, pas un déploiement production.
- **Réseau de scan.** La machine cible (Metasploitable) doit être joignable par le scanner. En labo : mets Greenbone (ou sa VM) et la cible sur le même réseau (host-only ou bridged sous VirtualBox).
- **Interface web sur https://127.0.0.1**, compte par défaut admin/admin → change le mot de passe immédiatement (la doc le rappelle).

> **Engagement de ce guide**
>
> Tous les outils, versions et besoins matériels de ce guide ont été vérifiés sur les sources officielles. Rien ici ne devrait te bloquer en cours de route pour cause de « licence surprise » ou de « machine trop faible ». Si un doute subsiste, il porte sur le temps de sync des feeds — jamais sur la puissance de ta machine.

3. Ce que tu construis vraiment

OpenVAS n'est pas le projet — c'est une brique, un simple fournisseur de données. Il te donne une chose : une liste de failles (CVE) avec un score de gravité (CVSS). Tout le reste, c'est toi qui le construis, et c'est là qu'est l'essentiel du travail et de la valeur technique.

Module	Qui le fait	Complexité
Inventaire des actifs	Toi (BDD + CRUD)	Faible
Import & parsing OpenVAS	Toi (parser XML)	Moyenne–Élevée
Moteur de scoring de risque	Toi (logique métier)	Élevée
Détection & remédiation auto	Toi (règles + triggers)	Moyenne–Élevée
Plan de traitement / suivi	Toi (BDD + workflow)	Moyenne
Dashboards & KPI	Toi (frontend + agrégation)	Moyenne
Génération de rapports PDF	Toi (ReportLab)	Moyenne
Le scan lui-même	OpenVAS (externe, prêt)	— (outil prêt)

> **Analogie**
>
> OpenVAS = le **thermomètre**. Il donne une mesure. Le vrai projet, c'est tout le système autour : le dossier du patient (inventaire), le diagnostic (scoring), la réaction automatique (détection + remédiation), le suivi et le rapport pour le médecin-chef.

4. Architecture générale

- **Source :** OpenVAS/Greenbone produit un rapport (export XML depuis l'interface web, ou via l'API GMP avec python-gvm).
- **Parser :** module Python (lxml) qui extrait CVE / CVSS / host, gère les cas manquants.
- **BDD :** actifs, vulnérabilités, scores, plans d'action, alertes, historique des scans.
- **Scoring :** combine gravité technique et contexte métier.
- **Détection + remédiation :** règles qui réagissent aux résultats du scoring (la brique de la prof).
- **Backend/API + Dashboard + Reporting :** orchestration, visualisation, génération PDF.

5. Stack technique (versions & licences)

Couche	Techno	Notes
Scanner	Greenbone CE (Docker)	Python 3.9+ pour l'API ; install via compose officiel
Backend	Python + FastAPI (ou Flask)	FastAPI = moderne, docs auto ; Flask = plus simple
Accès scan	python-gvm	pip install python-gvm ; ou export XML manuel
BDD	PostgreSQL (SQLite au début)	SQLite pour démarrer vite, migrer ensuite
Parsing	lxml / ElementTree	standard Python
Dashboard	Streamlit (rapide) ou React+Chart.js	Streamlit = un graphe en qq lignes
Rapport PDF	ReportLab + Matplotlib	graphes en images insérées dans le PDF
Conteneurs	Docker + docker-compose	reproductible, propre pour la démo
Hyperviseur	VirtualBox (si VM Linux)	PAS VMware (non open source)

> **Conseil rapidité**
>
> Pour un premier pipeline qui marche vite : **Flask/FastAPI + SQLite + Streamlit + ReportLab**, et import du rapport OpenVAS par **fichier XML** (plus simple que l'API GMP). Tu automatiseras via python-gvm et migreras vers PostgreSQL seulement une fois le cœur fonctionnel.

6. Modèle de données

Un modèle propre dès le départ évite de tout refactorer. Tables principales :

```
asset            (actif à protéger)
  id, name, ip_address, type, business_criticality (1-5),
  exposure (interne/externe), owner, created_at

vulnerability    (une faille trouvée par le scan)
  id, cve_id, cvss_score, title, description,
  severity, detected_at, source (openvas)

asset_vulnerability   (lien N–N + score contextualisé)
  id, asset_id (FK), vulnerability_id (FK),
  status (open/in_progress/fixed), risk_score

risk_treatment   (plan d'action / remédiation)
  id, asset_vulnerability_id (FK), action, owner,
  due_date, status, resolved_at, evidence, auto_created (bool)

alert            (détection : déclenchée automatiquement)
  id, asset_vulnerability_id (FK), level, message,
  created_at, acknowledged (bool)

scan             (historique des scans → tendance)
  id, scan_date, source, total_vulns, imported_at

framework_control  (optionnel : mapping ISO 27001 / NIST)
  id, control_id, framework, title, description
```

> **Astuce conception (le détail qui fait la différence)**
>
> Mets le **risk_score sur la table de liaison** asset_vulnerability, pas sur la vulnérabilité seule. C'est ce qui permet de dire « CVE-2023-X = risque 9/10 sur le serveur web exposé, mais 3/10 sur le poste interne ». Le champ **auto_created** sur risk_treatment et la table **alert** matérialisent la brique détection/remédiation automatique.

7. Le moteur de scoring de risque

C'est le cœur « intelligent » du projet, et ce qui le distingue d'un simple afficheur de scan. On ne se contente pas du CVSS brut (gravité technique) : on le pondère par le **contexte métier**.

```
Risk = CVSS_score × Criticité_actif × Facteur_exposition

  CVSS_score         = 0 à 10   (donné par OpenVAS)
  Criticité_actif    = 1 à 5    (défini dans ton inventaire)
  Facteur_exposition = 1.0 (interne) / 1.5 (externe/Internet)

→ normaliser sur 0–100, puis classer :
  Faible / Moyen / Élevé / Critique
```

Actif	CVSS	Criticité	Expo.	Risque	Niveau
Serveur web public	8.5	5	×1.5	≈ 88	**Critique**
Poste interne RH	8.5	2	×1.0	≈ 34	**Moyen**
Imprimante réseau	5.0	1	×1.0	≈ 10	**Faible**

Même faille (CVSS 8.5), trois risques très différents selon le contexte. **C'est exactement ce raisonnement qui fait la valeur GRC** : traduire un risque technique en priorité métier. Tu tries ensuite par risque décroissant → « on corrige quoi en premier cette semaine ? ».

> **Pour aller plus loin (optionnel)**
>
> Enrichir le score avec l'**exploitabilité** : existe-t-il un exploit public connu ? (bases EPSS ou CISA KEV, gratuites). À faire seulement après un pipeline de base fonctionnel.

8. Le pipeline OpenVAS

OpenVAS ne « connaît » pas l'entreprise : tu lui donnes des cibles (IP/hôtes), il scanne, identifie les versions de logiciels exposées, et les compare à sa base de tests (NVT) liés aux CVE.

8.1 — Installation (méthode officielle vérifiée : Docker)

```
# 1. Docker installé (voir doc officielle Docker Engine)
# 2. Dossier de travail
export DOWNLOAD_DIR=$HOME/greenbone-community-edition && mkdir -p $DOWNLOAD_DIR

# 3. Récupérer le compose officiel
curl -f -L https://greenbone.github.io/docs/latest/_static/compose.yaml \
     -o $DOWNLOAD_DIR/compose.yaml

# 4. Télécharger les images puis démarrer
docker compose -f $DOWNLOAD_DIR/compose.yaml pull
docker compose -f $DOWNLOAD_DIR/compose.yaml up -d

# (Un script "tout-en-un" officiel existe aussi :
#  setup-and-start-greenbone-community-edition.sh)

# 5. Définir le mot de passe admin (défaut = admin/admin, à changer !)
# 6. Interface web : https://127.0.0.1
```

> **Patience au 1er démarrage**
>
> Le premier lancement télécharge les feeds (VT, SCAP/CVE, CERT). Ça prend du temps (30 min → plusieurs heures) et de l'espace disque. **Lance-le tôt, laisse tourner.** Le scan ne sera vraiment exploitable qu'une fois les feeds chargés.

8.2 — Préparer des cibles (labo, en local)

Tu n'as pas de vrai réseau d'entreprise — c'est normal. On utilise des machines volontairement vulnérables :

- **Metasploitable 2** — VM Linux truffée de failles connues, idéale pour de vrais CVE.
- **DVWA** — application web vulnérable.
- Une vieille VM non patchée pour varier les résultats.

> **Cadre légal — impératif**
>
> On ne scanne **que ses propres machines de labo**, sur son propre réseau isolé. Scanner un système qui ne t'appartient pas est illégal. Metasploitable/DVWA sont faits exactement pour ça.

8.3 — Récupérer les résultats (2 options vérifiées)

- **Simple (recommandé pour démarrer) :** lancer le scan depuis l'interface web, exporter le rapport en **XML**, ton app importe le fichier.
- **Automatisé :** via l'API **GMP** et la lib **python-gvm** (get_reports avec le format_id XML) — tout piloté par code.

```
# Parsing du rapport XML (logique)
from lxml import etree
tree = etree.parse("rapport_openvas.xml")
for result in tree.iter("result"):
    host = result.findtext("host")
    name = result.findtext("name")
    cvss = result.findtext("severity")   # score CVSS
    cve  = extraire_cve(result)          # ta fonction (refs peuvent contenir 0..n CVE)
```

```
# → nettoyer, gérer les manquants, insérer en base,
#   relier à l'actif via l'IP
```

> **La réalité du parsing = la vraie valeur technique**
>
> Le XML n'est pas toujours propre : CVE parfois absents, un résultat = parfois plusieurs CVE, hôte à relier à l'inventaire par l'IP. Gérer ces cas proprement, c'est justement l'ingénierie de données qui donne du corps au projet.

9. Détection & remédiation automatique

C'est la brique demandée par l'encadrante. Objectif : que le système **réagisse tout seul** quand une situation critique apparaît, sans intervention manuelle. Implémentée au niveau applicatif, elle est parfaitement réaliste sur la durée du stage.

9.1 — La « détection »

Pas besoin de sondes réseau (Zeek/Suricata) ni d'IA. La détection = **corrélation automatique** à l'import d'un nouveau scan : le système repère les vulnérabilités dont le risque contextualisé dépasse un seuil *et* qui touchent un actif majeur. On peut aussi détecter une **nouveauté** (vuln critique absente du scan précédent = apparition d'une menace).

9.2 — La « remédiation automatique »

Dès qu'une détection se déclenche, le système exécute automatiquement une ou plusieurs actions :

- Créer automatiquement une **alerte** (table alert) avec niveau et message.
- Créer automatiquement une **action prioritaire** dans le plan de traitement (auto_created = true), assignée par défaut au responsable de l'actif.
- Envoyer une **notification** (email via SMTP, ou webhook) au responsable.
- Optionnel : marquer l'actif comme « à isoler » (statut), pour matérialiser l'idée de confinement sans monter de vrai réseau.

```
# Règle de détection + remédiation (logique)
SEUIL_CRITIQUE = 80
def evaluer(av):  # av = lien asset_vulnerability avec risk_score calculé
    if av.risk_score >= SEUIL_CRITIQUE and av.asset.business_criticality >= 4:
        creer_alerte(av, level="CRITIQUE")
        creer_action_auto(av, owner=av.asset.owner, priorite="haute")
        notifier(av.asset.owner, av)      # email / webhook
        # optionnel : av.asset.flag = "a_isoler"
```

> **Formulation honnête pour le rapport**
>
> Décris cette brique comme « **détection par corrélation et remédiation automatisée par règles** » — c'est exact et ça correspond au SOAR léger. Évite « détection de menaces par IA » : ce serait survendre. Si tu veux une touche IA, réserve-la à une fonction précise et honnête (ex. résumé auto d'alerte), en §13.

10. Planning semaine par semaine

Planning pour livrer un pipeline **complet et fonctionnel** en ~6 semaines. À la fin de chaque semaine, quelque chose marche. La brique détection/remédiation est intégrée en semaine 4.

Semaine 1 — Fondations, inventaire & install Greenbone

- Environnement (Python, BDD, Git, Docker) + dépôt Git.
- Modèle de données (tables asset, vulnerability, liaison, treatment, alert).
- Module inventaire des actifs : CRUD (criticité, exposition).
- Lancer l'install Greenbone en Docker + démarrer les feeds (tôt !).

Livrable : BDD prête + inventaire fonctionnel + feeds en cours de chargement.

Semaine 2 — Scan & import OpenVAS

- Préparer Metasploitable 2 sur réseau local isolé.
- 1er scan + export XML depuis l'interface web.
- Parser (lxml) : extraire CVE, CVSS, host ; gérer les manquants.
- Insérer en base + relier chaque vuln à l'actif (via IP).

Livrable : un scan réel importé et visible en base.

Semaine 3 — Moteur de scoring

- Formule Risk = CVSS × Criticité × Exposition.
- Calcul + stockage du risk_score sur chaque lien actif–vuln.
- Classement (Faible/Moyen/Élevé/Critique) + tri par priorité.

Livrable : chaque vulnérabilité a un score de risque contextualisé.

Semaine 4 — Détection & remédiation automatique

- Règles de détection (seuil + criticité actif + nouveauté vs scan précédent).
- Remédiation auto : création d'alerte + action prioritaire (auto_created).
- Notification (email SMTP ou webhook) au responsable.
- Vue « alertes » + « à corriger en priorité ».

Livrable : le système réagit seul à une vuln critique — brique clé de la prof.

Semaine 5 — Dashboard & KPI

- Dashboard : risques par niveau, heatmap, top actifs à risque, alertes.
- Courbe de tendance : risque global scan par scan (historique).
- KPI : % traité, risque moyen, nb de critiques ouverts, nb d'alertes.
- Filtres (actif, niveau, statut).

Livrable : dashboard interactif lisible.

Semaine 6 — Reporting & finitions

- Rapport PDF auto (technique + résumé exécutif) avec graphes.
- Nettoyage code, README, dockerisation propre.
- Test end-to-end : scan → import → score → détection → action → rapport.
- Documentation finale + captures pour le rapport de stage.

Livrable : projet complet, démontrable de bout en bout.

11. Dashboard, KPI & reporting PDF

KPI à afficher

- Score de risque global (jauge / gros chiffre).
- Vulnérabilités par niveau (Critique / Élevé / Moyen / Faible).
- % de vulnérabilités traitées + nb de critiques ouverts.
- Nombre d'alertes déclenchées (et non acquittées).
- Évolution du risque global dans le temps (scan par scan).
- Top 5 des actifs les plus exposés.

Visualisations utiles

- **Heatmap de risque** (criticité × gravité) — très parlante.
- **Barres / camembert** par sévérité.
- **Courbe temporelle** — la plus convaincante pour montrer une amélioration.
- **Tableau trié** par risque décroissant — répond à « quoi corriger d'abord ».

Le rapport PDF (le livrable qui compte le plus en GRC)

Auditeurs et direction ne voient pas ton code : ils voient ce document. Prévois deux formats à partir des mêmes données : un **rapport technique** (liste CVE/CVSS/actifs/statuts) et un **résumé exécutif** d'une page (posture globale, nb de critiques, tendance, top priorités, sans jargon). ReportLab (qui a généré ce guide) + Matplotlib pour les graphes suffisent.

> **Gain de temps**
>
> Avec **Streamlit**, un graphe interactif = quelques lignes. Idéal pour un dashboard fonctionnel vite. Ne consacre pas la moitié du projet au frontend — la valeur est dans le scoring, la détection et le reporting.

12. Pièges à éviter & bonnes pratiques

À faire

- **Pipeline de bout en bout d'abord** : 1 actif, 1 scan, 1 score, 1 alerte, 1 rapport — qui marche — avant d'enrichir. Évite l'effet « 5 modules à moitié faits ».
- **Lancer Greenbone très tôt** (semaine 1) à cause du temps de sync des feeds.
- **Git dès le jour 1**, commits fréquents.
- **Dockeriser** — démo reproductible, fait sérieux.
- **Documenter au fil de l'eau** (README + captures).

À éviter

- Regonfler le périmètre (ISO complet + IA + Zero Trust réseau...) → rien de fini.
- **Survendre la détection/remédiation** comme de l'« IA temps réel » : reste honnête (règles + corrélation).
- Passer trop de temps sur le frontend au détriment de la logique.
- Scanner des machines qui ne t'appartiennent pas (illégal).
- Négliger la gestion d'erreurs du parser (le XML réel est imparfait).
- Oublier de changer le mot de passe admin/admin de Greenbone.

> **La règle d'or**
>
> Un **pipeline complet et fini** (même modeste) vaut bien mieux qu'un projet ambitieux inachevé. La cohérence de bout en bout impressionne, et s'explique bien à un encadrant comme à un recruteur.

13. Extensions possibles

À n'attaquer que si le pipeline de base est solide et fini.

- **Mapping framework** (ISO 27001 / NIST CSF) : relier chaque faille à un contrôle (ex. « non-conformité au contrôle A.8.8 »). Transforme l'outil en véritable outil de conformité.
- **Exploitabilité** : enrichir le score via EPSS / CISA KEV (exploits publics connus).
- **OpenSCAP en 2■ source** : ajoute la dimension conformité de configuration (CIS Benchmark). Même architecture (scan → parse → score).
- **Rôles & permissions** : compte Analyste (tout) vs Direction (dashboard + PDF).
- **Assistant IA ciblé** : résumé automatique d'un rapport / d'une alerte, ou aide à la priorisation — sur UNE fonction précise, décrite honnêtement, pas « assister sur tout ».

> **OpenVAS vs OpenSCAP — à retenir**
>
> OpenVAS répond à « qu'est-ce qui est vulnérable ? » (CVE, risque technique). OpenSCAP répond à « est-on configuré comme prévu ? » (conformité CIS/STIG). Complémentaires — mais commence par OpenVAS seul.

14. Checklist de démarrage

Dans l'ordre, pour lancer le projet :

- ■ Installer Python, PostgreSQL (ou SQLite), Git, Docker (+ VirtualBox si VM Linux).
- ■ Créer le dépôt Git + structure du projet.
- ■ Écrire le schéma de base de données (tables clés).
- ■ Coder le module inventaire des actifs (CRUD).
- ■ Installer Greenbone CE via Docker et LANCER LES FEEDS (tôt !).
- ■ Monter Metasploitable 2 sur réseau local isolé.
- ■ 1er scan + export XML.
- ■ Écrire le parser XML et importer en base.
- ■ Implémenter la formule de scoring.
- ■ Ajouter la détection + remédiation automatique (règles + alerte + action).
- ■ Construire un dashboard minimal (Streamlit).
- ■ Générer le premier rapport PDF.
- ■ Tester le pipeline complet de bout en bout.

> **Prochaine brique concrète**
>
> Le meilleur point de départ : le **schéma de base de données** ou le **parser OpenVAS**. Une fois l'un des deux posé, tout le reste s'enchaîne.

Guide de projet — Plateforme GRC de gestion des vulnérabilités, enrichie par la détection des menaces et l'automatisation de la remédiation · Stack 100% open source · Faisabilité matérielle et licences vérifiées sur sources officielles · Périmètre pensé pour un stage de ~6 semaines à distance.